Chapter 3 – Methodology and work plan

Contents

3.1 Introduction 1
3.2 System Development Methodology 2
3.3 Requirement Gathering Techniques 3
3.4 Project Plan (Gantt Chart) 3
3.5 Development Tools and Technologies 4

3.1 Introduction

This chapter presents the methodology and work plan adopted for the development of the Intelligent Task Allocation System (ITAS). It explains the systematic approach followed to analyze requirements, design the system, implement core functionalities, and validate the final solution. Selecting an appropriate development methodology is critical to ensuring that project objectives are achieved within the defined academic timeline and scope. This chapter also outlines the requirement gathering techniques, project scheduling through a Gantt chart, and the tools and technologies used to build the system.

3.2 System Development Methodology

The Waterfall Model (Sequential Approach) was adopted as the system development methodology for the Intelligent Task Allocation System (ITAS). This model follows a structured and linear sequence of development phases, where each phase produces clearly defined outputs that serve as inputs for the next phase.

The selection of the Waterfall model is justified by the nature, scope, and objectives of the ITAS project for the following reasons:

First, the system addresses a well-defined problem domain with clearly identifiable functional requirements, including CV and portfolio parsing, task definition, skill extraction, suitability scoring, and dashboard visualization. These requirements are known in advance and do not require continuous redefinition during development, making a sequential methodology appropriate.

Second, the objectives of ITAS emphasize accuracy, consistency, and correctness in task–skill matching rather than rapid feature evolution. The Waterfall model supports this goal by enforcing a complete and validated requirements and design phase before implementation begins, reducing the risk of logical inconsistencies in the matching algorithms and scoring mechanisms.

Third, the system architecture of ITAS is modular and logically layered, consisting of distinct components such as the profile parser, matching engine, task manager, and dashboard. The Waterfall model aligns well with this structure by allowing each component to be designed, implemented, and tested in an orderly progression, ensuring system integrity and maintainability.

Finally, the project scope focuses on delivering a stable and fully functional prototype rather than continuous incremental releases. Since the system is intended

to demonstrate a complete end-to-end solution for intelligent task allocation, a methodology that prioritizes upfront analysis, detailed design, and controlled implementation is more suitable than iterative models such as Agile or Hybrid approaches.

Based on these considerations, the Waterfall development model provides a disciplined and systematic framework that aligns closely with the functional scope and technical objectives of the Intelligent Task Allocation System.

3.3 Requirement Gathering Techniques

Requirements for the Intelligent Task Allocation System were collected primarily through **brainstorming sessions** conducted among project team members. These sessions involved structured discussions to identify system objectives, user roles, functional requirements, and technical constraints.

Brainstorming sessions helped in:

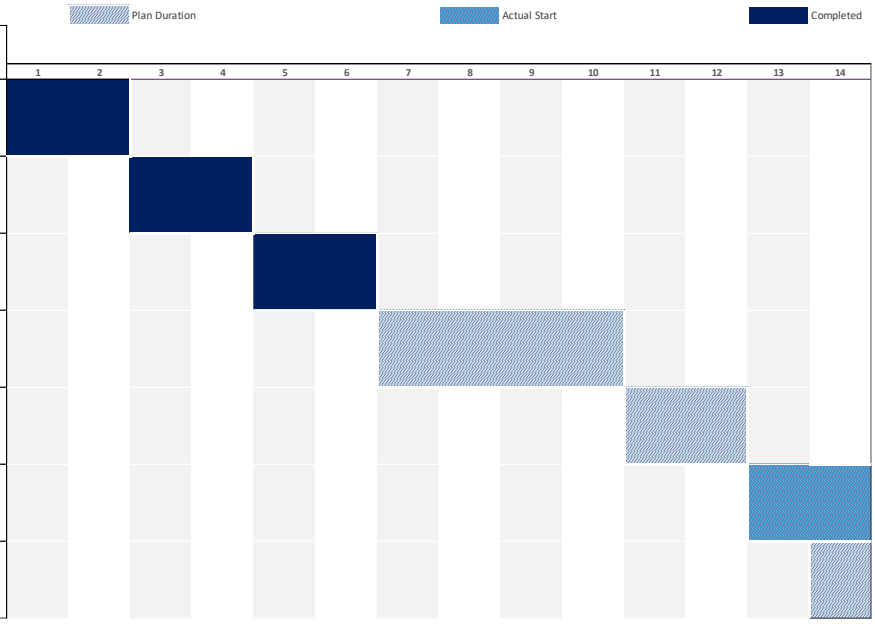
- Defining the core problem of manual and biased task allocation.
- Identifying system users such as Project Managers and Employees.
- Determining essential system functionalities such as CV parsing, task creation, skill matching, and dashboard visualization.
- Prioritizing features based on feasibility within the project timeline.

This collaborative approach allowed team members to share perspectives, refine ideas, and reach a unified understanding of system requirements. The outcomes of these sessions formed the foundation for the system analysis and design phases.

3.4 Project Plan (Gantt Chart)

Intelligent Task Allocation System (ITAS)

Phase No.	Project Phase	Activities Included	Duration	Weeks
1	Requirement Analysis	• Problem identification • Requirement gathering (brainstorming) • Scope definition	2 weeks	Week 1 – Week 2
2	Literature Review	• Review of existing systems • Comparative analysis • Research gap identification	2 weeks	Week 3 – Week 4
3	System Design	• System architecture design • Database design • Use case & workflow design	2 weeks	Week 5 – Week 6
4	Implementation	• Backend development (CV parsing, NLP, matching algorithm) • Frontend development (dashboard, task management) • Database integration	4 weeks	Week 7 – Week 10
5	Testing & Validation	• Functional testing • Algorithm validation • Bug fixing	2 weeks	Week 11 – Week 12
6	Documentation	• Final report writing • System documentation • User guide preparation	1.5 weeks	Week 13 – Mid Week 14
7	Final Submission & Presentation	• Final system review • Project submission • Presentation preparation	0.5 week	End of Week 14



3.5 Development Tools and Technologies

The following tools and technologies were used in developing the Intelligent Task Allocation System:

Programming Languages

- **Python:** Used for backend development and algorithm implementation due to its simplicity and strong support for data processing and NLP libraries.
- **JavaScript:** Used for frontend logic and interactive user interfaces.

Frameworks

- **Django or Flask:** Backend web framework used for handling routing, APIs, authentication, and business logic.
- **React.js:** Frontend framework used to build a responsive and dynamic dashboard interface.

Databases

- **PostgreSQL:** Chosen for its reliability, support for relational data, and ability to handle structured and semi-structured data efficiently.

Text Processing and NLP Tools

- **PyPDF2:** Used for extracting text from uploaded CV files.
- **spaCy or NLTK:** Used for Natural Language Processing tasks such as skill extraction and keyword identification.

Development Environments

- **Visual Studio Code (VS Code):** Used as the primary code editor due to its flexibility, extensions, and ease of use.
- **Git/GitHub:** Used for version control and collaborative development.

These tools were selected because they are open-source, well-documented, and suitable for building a scalable and maintainable academic software project.